# Agent Orchestration

## Decomposing Complex Features

**Day 1 - Session 1, Topic 1**

KavinSchool

# Prerequisites

Confirm these prerequisites:

- Experience using Claude Code or a similar AI coding assistant
- Working knowledge of Git branches and merges
- Comfort using a terminal to inspect files and run commands

# Introduce Yourself

In one chat message, type:

1. Your name
2. Your location or time zone
3. Your role
4. Your total experience in information technology (IT) and at your current organization

> **Chat Waterfall**
>
> **Type your answer in the chat, but do not send it yet.**
>
> Wait for the countdown: 3... 2... 1... BLAST
>
> You may share aloud when invited, keep a private note, or pass.

# Your Agent Orchestration Experience

In one chat message, type:

1. **Your current orchestration experience** (single-agent tasks, task planning, or parallel sessions)
2. **Your main focus** (development, technical leadership, or delivery)
3. **One orchestration challenge or question** you want answered by this course

> **Chat Waterfall**
>
> **Type your answer in the chat, but do not send it yet.**
>
> Wait for the countdown: 3... 2... 1... BLAST
>
> You may share aloud when invited, keep a private note, or pass.

# Your Agent Orchestration Experience: What We Learned

Your responses help us emphasize:

- **Your starting point:** From bounded single-agent tasks to coordinated parallel work
- **Your focus area:** Planning, implementation, technical leadership, or integration
- **Your key questions:** Task boundaries, model selection, handoffs, and verification

> **Compare your response with these themes and note one topic to revisit.**

# What We'll Cover

1. Agents, agentic development, and orchestration
2. Why complex features need decomposition
3. Agent-ready boundaries, contracts, and dependencies
4. Task cards, executable evidence, and demos
5. Lab 1.1: Agent-Ready Work Plan

# Two Demo Tracks: Choose Your Path

Choose the track that fits your role. The instructor marks the other track **SKIP**.

### APPLIED ORCHESTRATION

**Node.js / Claude-Helper**

Follow coding agents through a web project.

*Best for frontend and backend developers.*

### PYTHON AUTOMATION

**Scripted / Python**

Build orchestration harnesses with AST parsing and local models.

*Best for platform engineers.*

# Assessing Your Complexity Level

**BEGINNER TO INTERMEDIATE**

### Track 1: Applied Orchestration

Learners just getting started with agentic development.

*Practical application using a pre-built CLI tool.*

**ADVANCED**

### Track 2: Python Automation

Learners with a strong understanding of Python and orchestration.

*Building custom agent harnesses and direct model interactions.*

# Choose Your Track

Which track aligns best with your role and goals for this course?

**Type your choice (1 or 2):**

1. **Applied Orchestration** (Node.js / Claude-Helper)
2. **Python Automation** (Scripted Frameworks)

> **Chat Waterfall**
>
> **Type your answer in the chat, but do not send it yet.**
> Wait for the countdown: 3... 2... 1... BLAST
>
> This helps the instructor steer the upcoming demos to match the room's preference.

# What Is an Agent?

An agent works toward a goal by repeatedly deciding and acting.

1. It receives a goal, context, and constraints.
2. A model chooses the next action.
3. Tools inspect or change the environment.
4. Results become context for the next decision.
5. The loop stops when the goal is met or control returns to a human.

# Agentic Development in This Course

A coding agent can inspect a repository, edit files, and run commands.

It still needs a bounded assignment, relevant context, permitted tools, and a clear definition of done.

**Human responsibility:** Set intent, approve consequential decisions, and judge evidence.

**Agent responsibility:** Perform the assigned work and report results, checks, and unresolved risks.

# Skill vs Agent: Definitions

**REFERENCE ONLY**

| Extension | Primary purpose | Execution |
|---|---|---|
| Skill | Reusable instructions, knowledge, or workflow | Used by the current session or invoked explicitly |
| Agent | Specialized worker with its own context and operating boundaries | Runs an independent reasoning-and-action loop and returns a result |

# Skill vs Agent: When to Use

**REFERENCE ONLY**

**Use a skill when:** You want Claude to follow a repeatable procedure.

**Use an agent when:** The work benefits from separate context, specialized tools, permissions, or parallel execution.

> A skill can guide an agent, and a skill can be configured to run through a subagent. They are complementary, not competing features.

# What Is Orchestration?

Orchestration is the coordination of agents, tools, and people toward a shared outcome.

- Which work belongs to each agent
- What runs sequentially or in parallel
- Which context and artifacts move between tasks
- Where verification and human approval are required

> More agents do not create orchestration; explicit coordination does.

# Why Decomposition Comes First

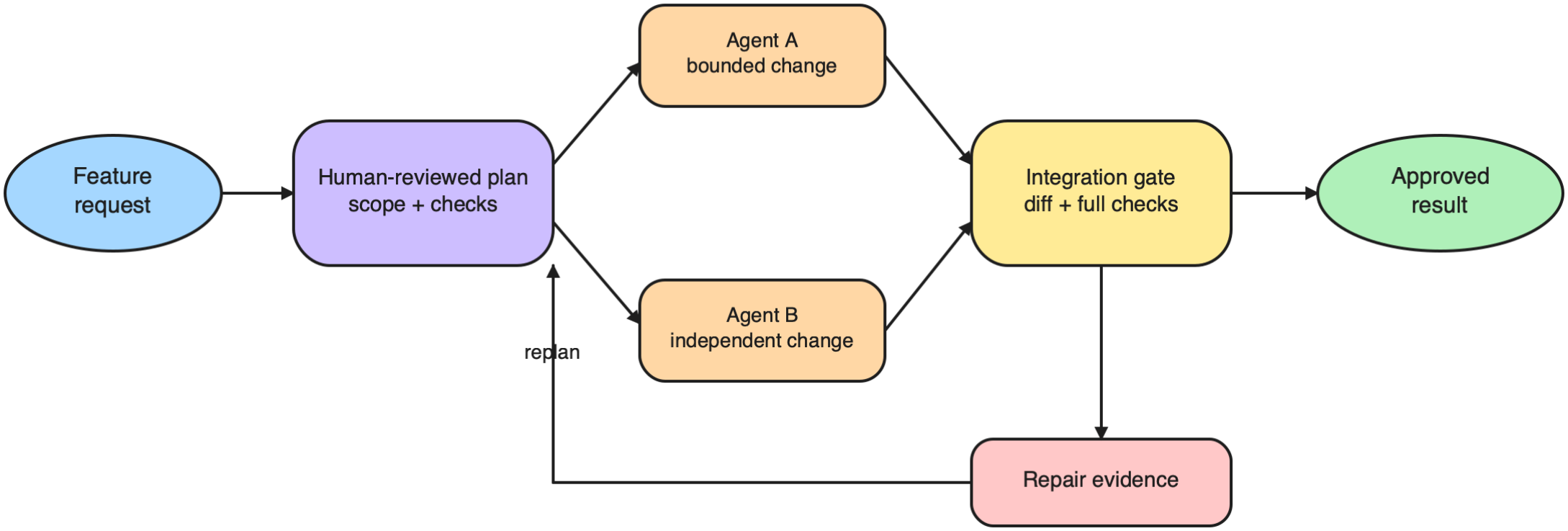
A feature is usually too broad to assign safely as one prompt.

Decomposition turns it into units with clear outputs, ownership, dependencies, and checks.

This enables parallel work where tasks are independent, limits the impact of failure, and makes progress verifiable.

> Before coordinating agents, define work that can be assigned and evaluated independently.

# Orchestration at a Glance




Parallel work earns its speed only when boundaries and checks are explicit.